ARCH TECHNOLOGIES

Sharpening your hidden skills for a brighter future

INTERNSHIP REPORT

Month 1 · Task 2

SecureMart – Secure E-Commerce App

Full Name: Yasir Rasheed

Internship Domain: Android App Development

Email Address: ygill1207@gmail.com

1. Introduction

SecureMart is a mobile-only e-commerce application for Android. Registered users can browse products, add them to a cart and buy them, and the same users can list their own products for sale. The task brief places the emphasis on data security and payment safety, so security is built into every layer of the app rather than added at the end.

The app is written in Kotlin with Jetpack Compose. All data is stored in a local Room (SQLite) database, so the project runs without a server. Payments go through a gateway interface with a demo implementation, which shows how a real payment provider would be connected without the app ever storing card details.

2. Objectives

- Provide a mobile-only shopping experience: browse, search, cart, checkout and order history.
- Allow only registered users to buy, and let every user list, edit and remove their own products.
- Protect user data: strong password storage, safe login, automatic sign-out and private storage.
- Make payments safe: validate cards, never store card numbers or security codes, and recalculate totals on the trusted side.
- Keep orders consistent: no overselling, and no order recorded without a successful payment.

3. Tools and Technologies

Item	Details
Language	Kotlin 1.9
IDE	Android Studio (Gradle Kotlin DSL, JDK 17)
UI	Jetpack Compose with Material 3, portrait phone layout
Database	Room 2.6 (SQLite) with Kotlin Flow and KSP
Architecture	MVVM: Compose screens, ViewModel, Repository, Room DAOs
Cryptography	PBKDF2WithHmacSHA256, SecureRandom, MessageDigest (javax.crypto)
Photos	Android photo picker (no storage permission needed), Bitmap re-encoding
SDK levels	minSdk 26 (Android 8.0), targetSdk 34 (Android 14)

4. Features

Feature	Description
Registration and login	Name, email and password; password strength rules; secure sign-in with lockout.
Shop	Product grid with pictures, live search and six category filters.

Feature	Description
Product details	Large photo, price, seller name, stock and quantity selector.
Cart	Change quantities, remove items and see the running total.
Checkout	Shipping details and card entry with validation, then a payment confirmation.
Sell tab	List a product with a photo, edit or remove it, and see sales of your own items.
Order history	Past orders with items, total, card brand and last four digits, and payment reference.
Profile	Account details and sign out.
Sample data	A demo seller with nine products and pictures is created on first launch.

5. Security and Payment Safety Design

This is the core of the task. The table lists each risk, the protection that was implemented and where it lives in the code.

Risk	Protection	Code
Stolen password data	Passwords are never stored. Each user has a random 16-byte salt and a PBKDF2-HMAC-SHA256 hash with 120,000 iterations.	PasswordHasher
Timing attacks	Hashes are compared with MessageDigest.isEqual, which takes the same time however many bytes match.	PasswordHasher.verify
Password guessing	After 5 wrong attempts the account is locked for 5 minutes.	Repository.login
Account enumeration	One generic message for wrong email or password; a dummy hash is computed for unknown emails so timing is the same.	Repository.login
Unattended phone	The session ends after 30 minutes away from the app.	SessionManager
Card data leaks	Card number and CVV exist only in memory during payment. Only brand, last four digits and a payment reference are saved. Card fields do not use saved state.	PaymentGateway, Order
Typing mistakes / invalid cards	Luhn checksum, expiry date check and CVV length (3, or 4 for Amex).	CardValidator
Price tampering	The total is calculated from prices in the database, never taken from the screen.	Repository.placeOrder
Overselling / double spend	Stock is reduced with an atomic conditional update inside a database transaction. If anything fails, the payment is voided.	placeOrder, ProductDao
Editing other people's items	Every edit and delete checks that the product belongs to the signed-in user; buying your own	Repository

Risk	Protection	Code
	product is blocked.	
SQL injection	All queries are parameterised through Room.	Daos
Bad input	Text is trimmed, control characters removed and lengths limited; email, price and stock are validated.	InputValidator
Photo privacy	Selected photos are re-encoded, which removes EXIF data such as GPS location.	ImageStore
Network and backup leaks	No INTERNET permission, cleartext traffic disabled and backups turned off, so data stays on the device.	AndroidManifest

Payment gateway. The app talks to payments only through the PaymentGateway interface. The included MockPaymentGateway moves no real money. In production a provider such as Stripe, JazzCash or Easypaisa would implement the same interface, so card details go directly to the provider and the app keeps only the returned reference.

Demo cards. 4242 4242 4242 4242 is approved and 4000 0000 0000 0002 is declined. Use any future expiry date and a 3-digit CVV.

6. System Design

6.1 Architecture

The Compose screens display state and report actions. MartViewModel holds the signed-in user and screen state and calls the Repository. The Repository contains all business rules (validation, ownership checks, checkout) and uses the DAOs. The security and payment packages are independent of the UI and easy to test.

Layer	Files
UI	MainActivity, SecureMartApp, AuthScreen, HomeScreen, ProductDetailScreen, CartScreen, CheckoutScreen, SellScreen, ProductFormScreen, OrdersScreen, ProfileScreen, Components, Theme
ViewModel	MartViewModel
Data	Entities, Daos, AppDatabase, Repository, ImageStore, ProductCategory, Models
Security	PasswordHasher, SessionManager, Validators (InputValidator, CardValidator), SecurityConfig
Payment	PaymentGateway, MockPaymentGateway

6.2 Database tables

Table	Main fields
users	id, name, email (unique), passwordHash, salt, failedAttempts, lockedUntil
products	id, sellerId, sellerName, title, description, priceCents, category, stock, imagePath, active

Table	Main fields
cart_items	id, userId, productId, quantity (unique per user and product)
orders	id, buyerId, totalCents, shipping fields, cardBrand, cardLast4, paymentRef, status
order_items	id, orderId, productId, sellerId, title, priceCents, quantity (snapshot at purchase time)

Prices are stored in cents as integers to avoid rounding errors, and each order item keeps a snapshot of the title and price so history stays correct if the seller later changes the listing.

6.3 Checkout flow

- The buyer enters shipping details and card details. Both are validated and cleaned.
- The cart is loaded from the database. Own products and quantities above stock are rejected.
- The total is calculated from database prices and sent to the payment gateway for authorisation.
- Inside one database transaction the stock of every item is reduced with a conditional update, the order and its items are saved and the cart is cleared.
- If the transaction fails (for example, an item just sold out) the payment is voided and the buyer sees a clear message. Nothing is half saved.

7. Implementation

This section explains the main parts of the code. The complete project is supplied as an Android Studio project.

7.1 Password hashing (security/PasswordHasher.kt)

A new random salt is created for every user and the password is stretched with 120,000 PBKDF2 iterations. The plain password is cleared from memory after hashing, and verification uses a constant-time comparison.

```
/**
 * Password storage: PBKDF2-HMAC-SHA256, 120,000 iterations, 16-byte random salt per user.
 * The plain password is never stored or logged.
 */
object PasswordHasher {
    private const val ITERATIONS = 120_000
    private const val KEY_BITS = 256

    fun newSalt(): ByteArray = ByteArray(16).also { SecureRandom().nextBytes(it) }

    fun hash(password: CharArray, salt: ByteArray): ByteArray {
        val spec = PBEKeySpec(password, salt, ITERATIONS, KEY_BITS)
        try {
            return
SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).encoded
        } finally {
            spec.clearPassword()
        }
    }

    fun encode(bytes: ByteArray): String = Base64.encodeToString(bytes, Base64.NO_WRAP)
```

```
    fun decode(text: String): ByteArray = Base64.decode(text, Base64.NO_WRAP)

    /** Constant-time comparison so timing does not reveal how many bytes matched. */
    fun verify(password: CharArray, saltB64: String, expectedB64: String): Boolean {
        val actual = hash(password, decode(saltB64))
        return MessageDigest.isEqual(actual, decode(expectedB64))
    }
}
```

7.2 Secure login and lockout (data/Repository.kt)

Unknown emails still trigger a hash so response time does not reveal which accounts exist. Wrong passwords are counted and the account is locked after the limit.

```
    suspend fun login(email: String, password: String): AuthResult {
        val cleanEmail = email.trim().lowercase(Locale.ROOT)
        val user = users.findByEmail(cleanEmail)
        if (user == null) {
            // Do the same amount of work as a real login so response time does not reveal
            // whether an account exists.
            withContext(Dispatchers.Default) { PasswordHasher.hash(password.toCharArray(),
ByteArray(16)) }
            return AuthResult.Error(GENERIC_LOGIN_ERROR)
        }

        val now = System.currentTimeMillis()
        if (user.lockedUntil > now) {
            val minutes = (user.lockedUntil - now) / 60_000 + 1
            return AuthResult.Error("Too many failed attempts. Try again in $minutes min.")
        }

        val ok = withContext(Dispatchers.Default) {
            PasswordHasher.verify(password.toCharArray(), user.salt, user.passwordHash)
        }
        if (ok) {
            if (user.failedAttempts != 0 || user.lockedUntil != 0L)
users.setLockState(user.id, 0, 0)
            return AuthResult.Success(user)
        }

        val failed = user.failedAttempts + 1
        return if (failed >= SecurityConfig.MAX_LOGIN_ATTEMPTS) {
            users.setLockState(user.id, 0, now + SecurityConfig.LOCK_MINUTES * 60_000)
            AuthResult.Error("Too many failed attempts. Account locked for $
{SecurityConfig.LOCK_MINUTES} minutes.")
        } else {
            users.setLockState(user.id, failed, 0)
            AuthResult.Error(GENERIC_LOGIN_ERROR)
        }
    }
```

7.3 Session timeout (security/SessionManager.kt)

The user id and the last-active time are kept in private app storage. MainActivity calls touch() when the app goes to the background and userId() when it returns; if more than 30 minutes have passed the user is signed out.

```
class SessionManager(context: Context) {
    private val prefs = context.applicationContext
        .getSharedPreferences("session", Context.MODE_PRIVATE)

    fun start(userId: Long) {
        prefs.edit()
            .putLong(KEY_USER, userId)
            .putLong(KEY_ACTIVE, System.currentTimeMillis())
            .apply()
    }

    /** Remember the moment the user left the app. */
    fun touch() {
        if (prefs.contains(KEY_USER)) {
            prefs.edit().putLong(KEY_ACTIVE, System.currentTimeMillis()).apply()
        }
    }

    /** Returns the signed-in user id, or null if nobody is signed in or the session timed
out. */
    fun userId(): Long? {
        val id = prefs.getLong(KEY_USER, -1L)
        if (id < 0) return null
        val last = prefs.getLong(KEY_ACTIVE, 0L)
        if (System.currentTimeMillis() - last > TIMEOUT_MS) {
            clear()
            return null
        }
        return id
    }

    fun clear() {
        prefs.edit().clear().apply()
    }

    companion object {
        const val TIMEOUT_MS = 30 * 60 * 1000L
        private const val KEY_USER = "user_id"
        private const val KEY_ACTIVE = "last_active"
    }
}
```

7.4 Card validation (security/Validators.kt)

Before any payment, the card number is checked with the Luhn algorithm, the expiry date must be in the future and the CVV must have the right length.

```
object CardValidator {

    fun digitsOnly(s: String): String = s.filter { it.isDigit() }
```

```
    fun brand(digits: String): String = when {
        digits.startsWith("4") -> "Visa"
        digits.length >= 2 && digits.substring(0, 2).toInt() in 51..55 -> "Mastercard"
        digits.length >= 4 && digits.substring(0, 4).toInt() in 2221..2720 -> "Mastercard"
        digits.startsWith("34") || digits.startsWith("37") -> "Amex"
        else -> "Card"
    }

    /** Luhn checksum: catches typing mistakes in card numbers. */
    fun luhnValid(number: String): Boolean {
        if (number.length < 13 || number.length > 19) return false
        var sum = 0
        var alternate = false
        for (i in number.length - 1 downTo 0) {
            var d = number[i] - '0'
            if (alternate) {
                d *= 2
                if (d > 9) d -= 9
            }
            sum += d
            alternate = !alternate
        }
        return sum % 10 == 0
    }

    fun problem(card: CardInput): String? {
        val digits = digitsOnly(card.number)
        if (!luhnValid(digits)) return "Card number is not valid"
        val month = card.expMonth.toIntOrNull()
        val year = card.expYear.toIntOrNull()
        if (month == null || month !in 1..12 || card.expYear.length != 2 || year == null) {
            return "Enter the expiry as MM and YY"
        }
        val now = Calendar.getInstance()
        val nowYear = now.get(Calendar.YEAR) % 100
        val nowMonth = now.get(Calendar.MONTH) + 1
        if (year < nowYear || (year == nowYear && month < nowMonth)) return "This card has
expired"
        if (year > nowYear + 20) return "Expiry year is not valid"
        val cvvLength = if (brand(digits) == "Amex") 4 else 3
        if (card.cvv.length != cvvLength || !card.cvv.all { it.isDigit() }) {
            return "Security code (CVV) must be $cvvLength digits"
        }
        return null
    }
}
```

7.5 Payment gateway (payment/PaymentGateway.kt)

The interface hides the payment provider. The demo gateway approves or declines a test card and returns only a reference, brand and last four digits.

```
interface PaymentGateway {
    suspend fun authorize(amountCents: Long, card: CardInput): PaymentResult
    suspend fun void(reference: String)
}
```

```
/** Demo gateway: no real money moves. */
class MockPaymentGateway : PaymentGateway {

    override suspend fun authorize(amountCents: Long, card: CardInput): PaymentResult {
        delay(1200) // pretend to talk to the bank
        val digits = CardValidator.digitsOnly(card.number)
        val brand = CardValidator.brand(digits)
        val last4 = digits.takeLast(4)
        if (digits == DECLINED_TEST_CARD) {
            return PaymentResult(false, "", brand, last4, "Your card was declined. Try another
card.")
        }
        val ref = "PAY-" + UUID.randomUUID().toString().replace("-",
"").take(12).uppercase(Locale.ROOT)
        return PaymentResult(true, ref, brand, last4, "Approved")
    }

    override suspend fun void(reference: String) {
        delay(300) // a real gateway would release the held funds here
    }

    companion object {
        const val DECLINED_TEST_CARD = "4000000000000002"
    }
}
```

7.6 Safe checkout (data/Repository.kt)

This function is the heart of payment safety: validate, recompute the total, authorise the payment, then save everything in one transaction, voiding the payment if the save fails.

```
    suspend fun placeOrder(buyer: User, ship: ShippingInfo, card: CardInput): CheckoutResult {
        val shipping = ShippingInfo(
            name = InputValidator.cleanLine(ship.name, 60),
            phone = InputValidator.cleanLine(ship.phone, 20),
            address = InputValidator.cleanLine(ship.address, 150),
            city = InputValidator.cleanLine(ship.city, 40)
        )
        InputValidator.shippingProblem(shipping)?.let { return CheckoutResult.Error(it) }
        CardValidator.problem(card)?.let { return CheckoutResult.Error(it) }

        val lines = cart.lines(buyer.id)
        if (lines.isEmpty()) return CheckoutResult.Error("Your cart is empty")
        for (line in lines) {
            if (line.sellerId == buyer.id) return CheckoutResult.Error("Remove your own
product from the cart")
            if (line.quantity > line.stock) {
                return CheckoutResult.Error("Only ${line.stock} of \"${line.title}\" left in
stock")
            }
        }
        // The amount is always calculated from database prices, never taken from the screen.
        val total = lines.sumOf { it.priceCents * it.quantity }

        val payment = gateway.authorize(total, card)
        if (!payment.approved) return CheckoutResult.Error(payment.message)
```

```
        return try {
            val orderId = db.withTransaction {
                val fresh = cart.lines(buyer.id)
                if (fresh.isEmpty()) throw OrderException("Your cart changed. Please review
it.")
                var sum = 0L
                for (line in fresh) {
                    if (products.decrementStock(line.productId, line.quantity) == 0) {
                        throw OrderException("\"${line.title}\" just sold out. Please review
your cart.")
                    }
                    sum += line.priceCents * line.quantity
                }
                if (sum != total) throw OrderException("Prices changed. Please review your
cart.")

                val id = orders.insertOrder(
                    Order(
                        buyerId = buyer.id,
                        totalCents = sum,
                        shipName = shipping.name,
                        shipPhone = shipping.phone,
                        shipAddress = shipping.address,
                        shipCity = shipping.city,
                        cardBrand = payment.brand,
                        cardLast4 = payment.last4,
                        paymentRef = payment.reference,
                        status = "PAID"
                    )
                )
                orders.insertItems(
                    fresh.map {
                        OrderItem(
                            orderId = id,
                            productId = it.productId,
                            sellerId = it.sellerId,
                            title = it.title,
                            priceCents = it.priceCents,
                            quantity = it.quantity
                        )
                    }
                )
                cart.clear(buyer.id)
                id
            }
            CheckoutResult.Success(orderId)
        } catch (e: OrderException) {
            gateway.void(payment.reference) // release the payment if the order could not be
saved
            CheckoutResult.Error(e.message ?: "Could not place the order")
        }
    }
```

7.7 Listing a product (data/Repository.kt)

The seller's input is cleaned and validated. Editing is allowed only when the product belongs to the signed-in user, and a newly picked photo is re-encoded by ImageStore before saving.

```
    suspend fun saveProduct(seller: User, productId: Long?, form: ProductFormData): String? {
        val title = InputValidator.cleanLine(form.title, 80)
        if (title.length < 3) return "Title must be at least 3 characters"
        val description = InputValidator.cleanText(form.description, 600)
        if (description.length < 10) return "Description must be at least 10 characters"
        val cents = InputValidator.parsePriceToCents(form.price)
            ?: return "Enter a valid price (for example 1250 or 1250.50)"
        val stock = form.stock.trim().toIntOrNull()?.takeIf { it in 0..999 }
            ?: return "Stock must be a number from 0 to 999"
        val category = ProductCategory.fromName(form.category).name

        val existing = if (productId != null) products.findById(productId) else null
        if (productId != null && (existing == null || existing.sellerId != seller.id)) {
            return "You can only edit your own listings"
        }

        var imagePath = existing?.imagePath
        if (form.imageUri != null) {
            val saved = withContext(Dispatchers.IO) { ImageStore.saveProductImage(appContext,
form.imageUri) }
                ?: return "Could not read that image. Please pick another one."
            withContext(Dispatchers.IO) { ImageStore.delete(imagePath) }
            imagePath = saved
        }

        if (existing == null) {
            products.insert(
                Product(
                    sellerId = seller.id,
                    sellerName = seller.name,
                    title = title,
                    description = description,
                    priceCents = cents,
                    category = category,
                    stock = stock,
                    imagePath = imagePath
                )
            )
        } else {
            products.update(
                existing.copy(
                    title = title,
                    description = description,
                    priceCents = cents,
                    category = category,
                    stock = stock,
                    imagePath = imagePath
                )
            )
        }
        return null
    }
```

7.8 Manifest hardening (AndroidManifest.xml)

The app requests no permissions at all, disables cleartext traffic and backups and is locked to portrait mode, as it is a mobile-only app.

```
<!-- No INTERNET permission: the app works fully offline and sends no data anywhere. -->
<application
    android:allowBackup="false"
    android:usesCleartextTraffic="false"
    android:theme="@style/Theme.SecureMart">
    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:screenOrientation="portrait"
        android:windowSoftInputMode="adjustResize" />
</application>
```

7.9 Optional screen protection (MainActivity.kt)

A single switch in SecurityConfig turns on FLAG_SECURE, which blocks screenshots, screen recording and the recent-apps preview. It is off in this build so that screenshots can be taken for this report.

```
if (SecurityConfig.BLOCK_SCREENSHOTS) {
    window.setFlags(WindowManager.LayoutParams.FLAG_SECURE,
                    WindowManager.LayoutParams.FLAG_SECURE)
}
```

8. Screenshots

The figures below show the working application running on a Pixel 7 emulator (Android API 36.1) in Android Studio. The sample product pictures bundled with the app are shown first.



Figure 1: Sample product pictures included in the app

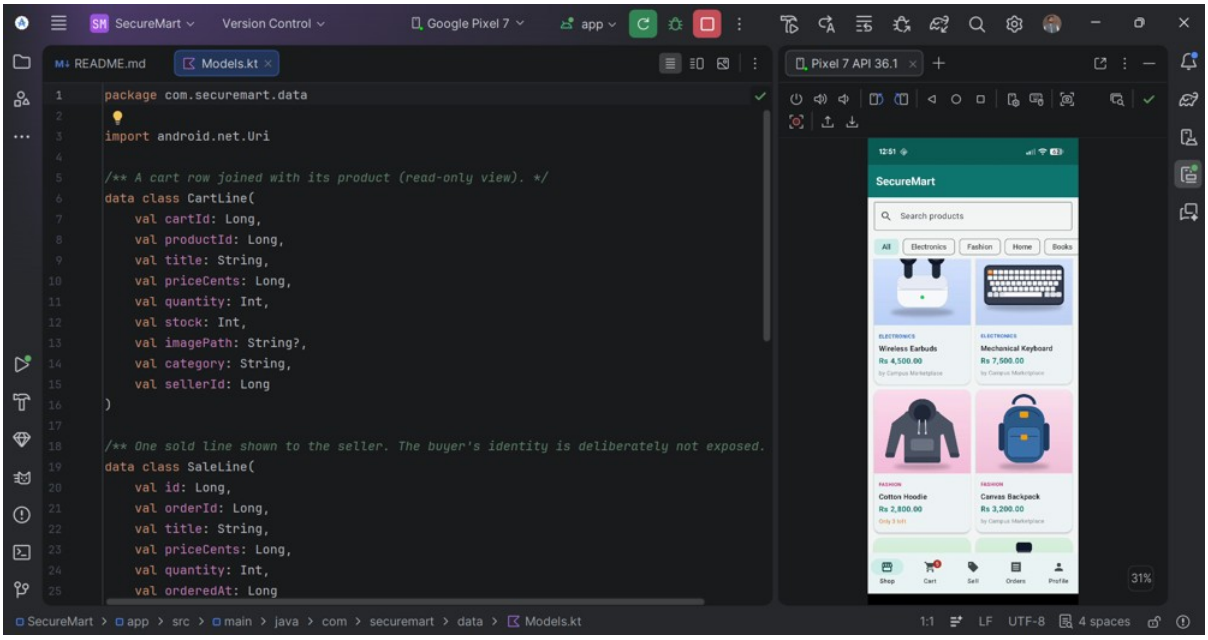


Figure 2: SecureMart running on the Pixel 7 emulator inside Android Studio

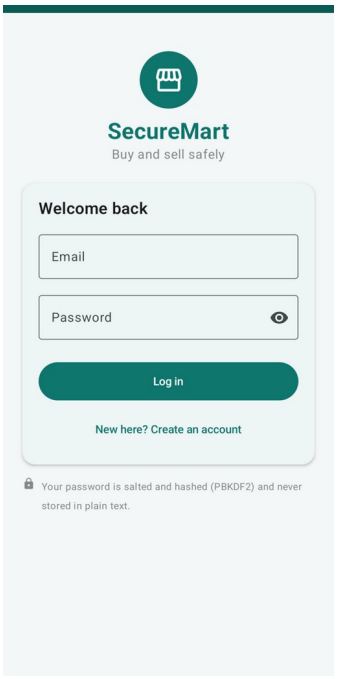


Figure 3: Login screen with password-protection note

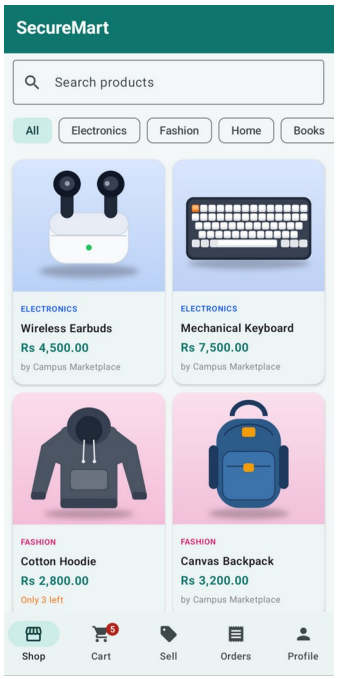


Figure 4: Shop with product pictures, search and category filters

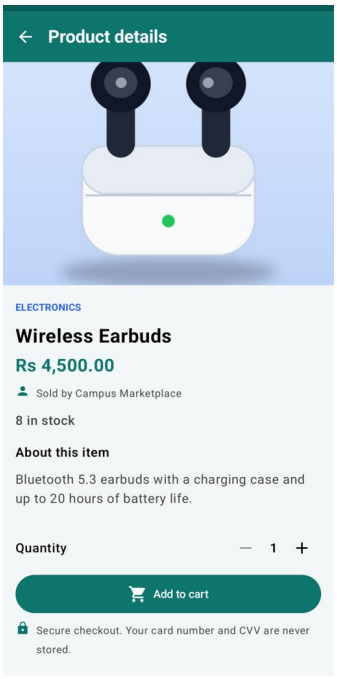


Figure 5: Product details with stock, quantity and Add to cart

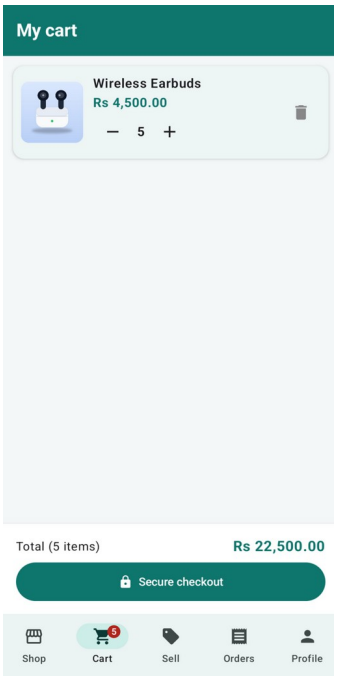


Figure 6: Cart with quantity stepper, total and Secure checkout

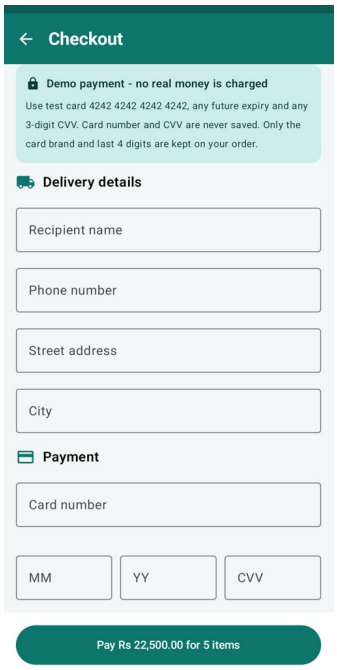


Figure 7: Checkout: delivery details and card entry (demo payment)

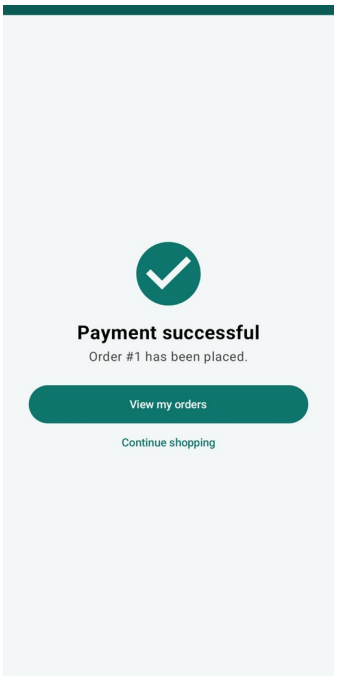


Figure 8: Payment successful and order placed

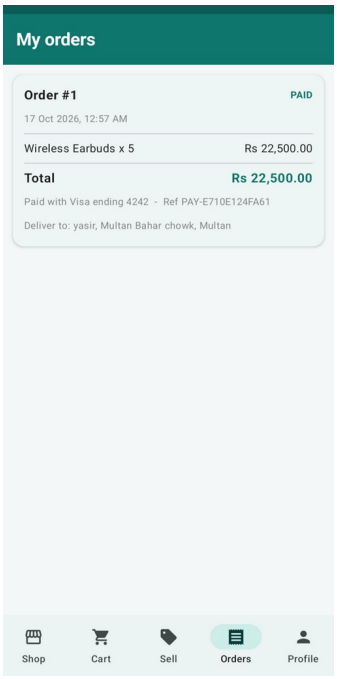


Figure 9: Order history: card brand, last 4 digits and payment reference only

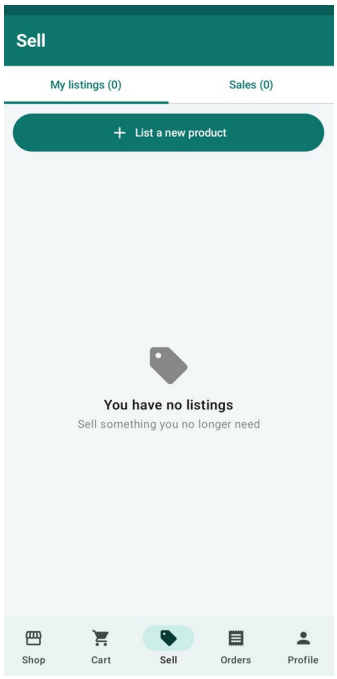


Figure 10: Sell tab: My listings and Sales

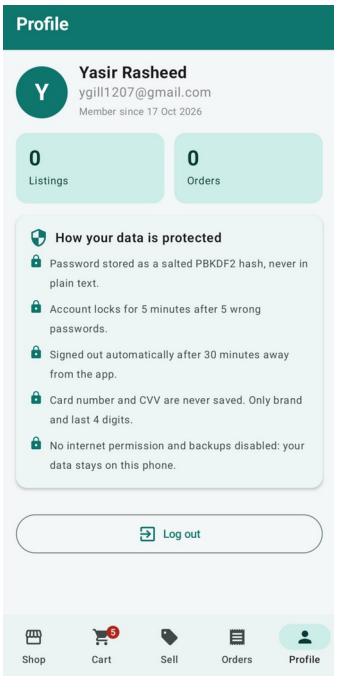


Figure 11: Profile showing how data is protected

9. Test Plan

No.	Test	Expected result
1	Register with a weak password	Message explains the password rules; no account is created
2	Register an email that already exists	Message that the account exists
3	Log in with a wrong password 5 times	Account locks for 5 minutes with a clear message
4	Log in with an unknown email	Same generic message as a wrong password
5	Open the shop, search and use category chips	Products with pictures; list filters correctly
6	Add a product to the cart and change its quantity	Total updates; quantity cannot exceed stock
7	Check out with card 4242 4242 4242 4242	Order is placed, stock reduced, cart emptied
8	Check out with card 4000 0000 0000 0002	Card declined; no order and stock unchanged
9	Enter an invalid card number or expired date	Validation message before any payment
10	View order history	Order shows card brand, last 4 digits and payment reference only
11	List a product with a photo from the Sell tab	Product appears in the shop with its photo
12	Try to buy your own product	Blocked with a message

No.	Test	Expected result
13	Leave the app for over 30 minutes	Signed out when the app is opened again

10. Conclusion and Future Work

SecureMart delivers a complete mobile shopping and selling experience with security as the main design goal. Passwords are salted and hashed, logins are protected against guessing, card data is never stored, totals are verified from trusted data and orders are saved atomically with payment rollback. The project covered Compose UI, Room, MVVM, cryptography, input validation and transactional logic.

Future work: connect a real payment provider through the existing gateway interface, add a cloud backend (for example Firebase Authentication and Firestore) so products and orders are shared between devices, add biometric login, product reviews, seller dashboards and delivery tracking.

11. How to Run

- Install Android Studio with JDK 17 and open the SecureMart folder.
- Wait for Gradle sync, then choose an emulator (API 26 or higher) or a connected phone.
- Press Run. Sign up with any email, or browse the sample products listed by the demo seller.
- Use the demo cards from Section 5 to test successful and declined payments.